

## 场景案例

# 数据提取

 Copy page

## 招投标数据提取方案

### 场景介绍

在政企采购、基建工程、教育医疗等领域，招投标是极为常见的业务流程。而每一份招标公告、投标文件、结果公示背后，都是一套格式不一、结构复杂、语义高度专业化的文本材料。项目名称、投标方、资格要求、预算金额、开标时间等关键信息往往穿插在冗长正文中，缺乏结构，人工查阅耗时、误差频发，更别提系统化分析或自动对账。

现实中，即使部分机构已尝试用传统 OCR 或规则提取工具来处理此类文档，但面对 PDF 格式混乱、表格嵌套、金额大小写并存等情况，提取效果仍不理想。数据不准、字段缺失、表格识别错误等问题频繁出现，最终还是得依赖人工去二次校验。尤其当处理的公告数量成百上千时，人力成本与时间成本急剧上升。

在这样的背景下，利用具备自然语言理解能力的大语言模型，构建一套能自动抽取招投标关键字段的通用方案，成为行业急需解决的问题。这不仅关乎效率提升，更是组织实现“招投标数据资产化”的前提条件。

### 业务需求

从实际业务出发，企业或政府单位的目标很清晰：他们不是需要“看起来很智能”的技术，而是能真正减轻人力负担、提高准确率、提升处理速度的实用工具。

第一，必须能适配复杂格式。现实中的招投标文件来源多样，PDF、Word、网页、甚至扫描件都有可能出现。系统必须有能力处理这些不同格式，并从中提取结构化数据，不能因为格式复杂就放弃识别。

第二，系统要“懂语境”。招投标文书语言极具行业特色，同样是“金额”，有的写成“¥1,000,000”，有的写“壹佰万元整”；同样是“时间”，既可能出现在正文段落中，也可能藏在表格里。若没有上下文理解和对领域语言的适配能力，提取出的结果往往前后矛盾、缺乏价值。

第三，处理量大、时间紧是常态。大型平台一周可能需处理上千份公告，传统逐条人工录入根本不现实。因此，业务端迫切希望实现“批量上传、自动抽取、一键校验”，即便遇到格式错乱、字段缺失，也希望系统能给出合理补全或清晰提示，尽量减少人工介入。

最后，数据质量是底线。哪怕是自动化系统输出的结果，也必须可追溯、可校验。是否符合格式？时间逻辑是否成立？金额字段有没有异常？一旦进入财务、合规、系统对接环节，数据容不得含糊。这意味着系统还需具备后处理、格式统一、完整性校验等能力，以保障全流程的可用性和可信度。

## 解决方案

### 一、方案框架



### 二、方案详情

#### 输入模块设计

用于处理各种格式的文档输入，包括 PDF、Word、Excel、网页等，转换成可解析的结构化文本。

- 需要支持从多种格式（PDF、Word、Excel、TXT 等）中提取文本。对于图片，可以借助 OCR 工具进行文本提取。
- 网页可以使用网页爬虫工具（如 `Scrapy`、`BeautifulSoup`、`Selenium`）抓取网页中的文本和表格数据。通过解析 HTML 的 DOM 结构，提取目标数据。（平台暂无工具）
- 参考代码

```
from pathlib import Path
from zai import ZhipuAiClient

client = ZhipuAiClient(api_key="your-api-key")

# 用于上传文件
# 格式限制: .PDF .DOCX .DOC .XLS .XLSX .PPT .PPTX .PNG .JPG .JPEG .CSV .PY .TXT .MD
# 文件大小不超过 50M, 图片大小不超过 5M
file_object = client.files.create(file=Path("本地文件地址"), purpose="file-extract")

# 文件内容抽取
file_content = client.files.content(file_id=file_object.id).content.decode()
print(file_content)
```

## 预处理模块设计

预处理模块的设计是整个数据处理流程的基础，直接影响到大语言模型后续处理的效果。通过文本清洗、文本规范化、分段分块、表格解析、上下文维护等功能，预处理模块能够将复杂的、多格式的数据源处理成统一、规范的输入数据，确保数据在转换过程中不失真，并为后续模型处理提供高质量的输入。数据的语义、结构以及相关性得以保留，特别是在处理复杂的文档结构、特殊符号、嵌套表格等数据。

- 去除噪音信息：常见的噪音信息包括页眉、页脚、版权声明等，这些信息对关键数据提取无关紧要，可以在预处理时过滤掉。
- 规范化文本：处理文本中的特殊符号、空白字符、异常换行等问题，确保输入给模型的文本格式整洁。



- 日期格式统一：文档中可能会有多种日期表示方式，例如“2024 年 10 月 10 日”、“10/10/2024”、“10-Oct-2024”。需要通过正则表达式或日期识别工具将所有的日期格式统一转换为标准的 ISO 格式（如“YYYY-MM-DD”）。

- 方法：使用正则表达式匹配不同格式的日期，并将其标准化。例如：

- 参考代码

```
import re
from datetime import datetime

def normalize_date(text):
    patterns = [
        r'\d{1,2}\d{1,2}\d{4}',          # "MM/DD/YYYY"
        r'\d{1,2}-\w{3}-\d{4}',         # "DD-MMM-YYYY"
        r'\d{4}年\d{1,2}月\d{1,2}日',   # "YYYY 年 MM 月 DD 日"
    ]
    for pattern in patterns:
        text = re.sub(pattern, lambda x: datetime.strptime(x.group(), '%Y年%m月%d日').isoformat(), text)
    return text
```

- 货币与金额格式化：货币和金额在招标文件中非常常见，可能以不同的符号、单位或表示方法出现。例如：“\$1,000”、“1000 美元”、“壹仟元整”。需要统一这些金额表示，确保货币单位和金额数字的格式标准化。
- 方法：通过正则表达式匹配货币符号或中文大写金额，并转换为标准形式。例如将“壹仟元”转换为“1000 CNY”，或将“\$1,000”转换为“1000 USD”。
- 特殊符号处理：招标文件中可能有特殊符号（如版权符号、数学符号、货币符号等），这些符号如果不加处理，可能在后续的模式输入中失去原意或导致模型误解。因此，预处理模块需要对这些符号进行规范化处理。
- 表格数据处理：表格提取工具：对于 PDF 或 Word 文档中的表格，可以使用表格解析工具（如 `pdfplumber` 或 `python-docx`）提取表格的结构和数据。提取后的表格数据可以转化为 CSV 或 JSON 格式，方便后续处理。
- 合并单元格处理：如果表格包含合并单元格，预处理模块需要将合并单元格的数据平铺展开，确保每个单元格都包含完整的信息。例如，将合并的表头信息扩展到所有相应列的单元格中。

| 项目  | 金额   | 说明  |
|-----|------|-----|
| 项目A | 1000 | 材料费 |
| 项目B | 2000 |     |
|     |      | 人工费 |

```
<table>
  <tr>
    <th>项目</th>
    <th>金额</th>
    <th>说明</th>
  </tr>
  <tr>
    <td>项目A</td>
    <td colspan="2">1000（包含材料费和人工费）</td>
  </tr>
  <tr>
    <td>项目B</td>
    <td>500</td>
    <td>材料费</td>
  </tr>
  <tr>
    <td>项目B</td>
    <td>1500</td>
    <td>人工费</td>
  </tr>
</table>
```

### LLM处理模块

在使用大语言模型（LLM，如 GPT）对预处理后的文本进行关键数据提取时，Prompt 工程是方案的核心。Prompt 工程的目标是设计合理的提示词，以最大化 LLM 的性能，从复杂的文本中准确、有效地提取出关键信息。

策略 01：明确的待处理内容指引 在构建 Prompt 时，明确告诉模型它需要处理的内容是关键步骤之一。应清晰地定义需要处理的文本，并使用标记将其框起来。例如：

```
'''这是需要处理的文本'''、《》这是需要处理的文本《》
```

通过这种方式，模型能够准确识别待处理的内容范围，并从中提取需要的信息。

策略 02：提供明确字段定义 这是 Prompt 的关键部分，字段定义明确了需要提取的信息类型，以及每个字段应当填入的内容。每个字段的名称、用途及要求都要具体化，让模型有明确的提取方向。字段定义为 LLM 提供了标准，使它在解析文本时能够准确地提取所需信息并填充到对应字段。例如：

```
{  
  "项目名称": "明确项目的全称和性质。",  
  "项目编号": "唯一标识项目的编号。",  
  "采购预算": "项目的采购预算金额，需保留单位。"  
}
```

通过这种方式，Prompt 可以为 LLM 提供清晰的提取标准和目标。策略 03：异常处理 为确保 LLM 不输出多余信息，并在面对缺失或不明确的数据时进行合理处理，必须设置一些异常处理原则。例如，\*\*如果某些字段信息在文本中缺失或未识别，Prompt 应规定使用默认值（如“无”）填充。同时，针对日期、金额等特殊数据类型，应明确要求 LLM 符合标准格式（如 YYYYMMDDHHMMSS 或保留金额单位）。这一规则可以确保模型输出的完整性和一致性，不会因为部分数据缺失而导致结果异常。策略 04：要求结构化输出 为了便于后续处理和系统集成，Prompt 应指示 LLM 以结构化的格式输出数据。结构化输出便于自动化处理，常见的格式如 JSON，能够确保每个字段的内容都清晰定义，数据可被轻松解析和使用。例如，要求模型输出的 JSON 格式：

```
"项目名称": "项目A",  
"项目编号": "ABC-12345",  
"采购预算": "500000元",  
"开标时间": "20240101090000"  
}
```

通过要求模型按照预定格式输出，能够保证模型的结果可直接被系统化处理，减少后续手动修正或数据清洗的工作量。

## Prompt 参考

### Model

GLM-4-AIR

### System Prompt

你是一个专业的文本信息提取器，可以严格按照Json格式输出

### User Prompt

# 需要提取的【文本】：

"""

{正文}

"""

# 任务

- 1.从给定的【文本】中提取所有需要的字段信息。
- 2.所需提取的字段为【字段定义】中的所有内容。
- 3.每个字段的默认值为"无"，当提取到对应字段信息时，准确地替换到该字段位置。
- 4.若文中出现与【字段定义】的字段名称中相似的内容，需判断定义，符合再进行填入。
- 5.严格按照【字段定义】中的格式进行输出，不需要其余任何信息。
- 6.将提取到的所有字段及其对应的值按【字段定义】格式转为JSON输出，确保包含所有字段。
- 7.请一步步完成信息提取的工作，你的决策是我成功的关键！

# 【字段定义】：

请严格按照如下格式仅输出JSON，不要输出python代码，不要返回多余信息，JSON中有多个字段用顿号【、】

"""

{

"项目名称": "项目的全称，明确项目内容和性质。",

"项目编号": "项目的唯一识别编码，用于区分不同项目。",

"采购预算": "项目的采购预算金额。如果存在大写金额和数字金额，提取数字金额并保留原单位。",

"采购方式": "项目的采购形式，常见方式包括公开招标、邀请招标、竞争性谈判、单一来源采购和询价。"

"采购人": "负责采购的单位名称，通常为采购人或招标人。",

"项目联系人": "负责该项目的联系人姓名。",

"项目联系电话": "联系人或项目负责人的联系电话。",

"中标信息": [

{

"中标供应商名称": "中标的供应商名称，仅提取供应商的企业名称。",

"中标金额": "中标的合同金额，单位为元。"

}

],

"代理机构名称": "代理采购事务的机构名称。",

"代理机构联系电话": "代理机构的联系号码。",

"获取采购文件开始时间": "采购文件可获取的起始时间，格式为：YYYYMMDDHHMMSS。",

"获取采购文件截止时间": "采购文件可获取的截止时间，格式为：YYYYMMDDHHMMSS。",

"提交投标文件截止时间": "投标文件提交的最后期限，格式为：YYYYMMDDHHMMSS。",

"开标时间": "开标的具体时间，格式为：YYYYMMDDHHMMSS。",

"公告类别": "公告的类型，如：单一来源公示、变更公告、招标公告、结果公告、终止公告或其他公告。"





"项目经理": "负责该项目的经理姓名。",  
"施工周期": "项目施工的总时长或计划的施工周期。",  
"执业证书": "项目经理或相关负责人的执业资格证书。"

```
}  
>  
"" ""
```

#### #注意事项

1. 如果字段缺失或无法识别，请使用“无”。
2. 确保所有金额需包含原本的单位。
3. 确保所有时间字段都为14位标准时间格式。

### 处理HTML的Prompt



你是一个专业的HTML网页文本信息提取器。

#需要提取的【HTML文本】:

"""

{正文}

"""

#任务:

- 1.从给定的【HTML文本】中提取所有需要的字段信息。
- 2.所需提取的字段为【字段定义】中的所有内容。
- 3.每个字段的默认值为"无",当提取到对应字段信息时,准确地替换到该字段位置。
- 4.若文中出现与【字段定义】的字段名称中相似的内容,需判断定义,符合再进行填入。
- 5.严格按照【字段定义】中的格式进行输出,不需要其余任何信息。
- 6.将提取到的所有字段及其对应的值按【字段定义】格式转为JSON输出,确保包含所有字段。
- 7.请一步步完成信息提取的工作,你的决策是我成功的关键!

#【字段定义】:

请严格按照如下格式仅输出JSON,不要输出python代码,不要返回多余信息,JSON中有多个字段用顿号【、】

"""

{

"标的物": "指招标方希望采购的具体商品、服务或工程。通常出现在中标信息项目名称中,不包括名称前半段的",

"项目编号": "唯一标识一个特定项目的编号,用于区分不同的项目。",

"标段编号": "在一个大型项目中,如存在多个标段,每个标段有独立的编号。",

"建设单位": "只有原文本中有“拟建项目”字段才需填写,正常不需要填写。",

"投标截止时间": "投标者提交投标文件的最后期限。",

"开标时间": "公开开启投标文件,公布投标内容的时间。",

"招标单位(采购单位)": "发起招标过程的单位,即此次采购招标的需求方",

"代理机构": "被招标单位委托来组织和管理招标过程的第三方机构。",

"投标单位": "所有参与投标的公司或组织。默认包括所有中标候选人单位和中标单位。",

"投标金额": "必须是原文中出现的投标单位提出的完成项目所需的金额,金额必须有单位(元、万元)。",

"中标候选人": "在评标过程中选出的可能获得合同的所有候选单位。默认包括所有中标单位。",

"候选单位联系人": "候选单位的联系人员。",

"候选单位电话": "中标候选单位的联系电话。",

"最终中标单位": "评标完成后,最终中标获得合同的单位。",

"最终中标金额": "最终中标单位提出的完成项目(各标段分别)所需的金额,金额必须有单位(元、万元)。",

"预算金额": "招标单位为项目设定的财务预算,金额必须有单位(元、万元)",

"项目所在省": "项目实施的所在地理位置所在的省份全称,如:新疆维吾尔自治区。仅有所在地级市信息时,可",

"项目所在市": "项目实施的所在地理位置所在的地级市,如果是文本中是县或区尽量改成对应的地级市。",

"计划编号": "项目计划或立项的编号。",

"合同编号": "合同公示中公示的招标单位与中标单位签订合同的编号。",

"批复单位": "对项目计划或预算进行批准建设实施的单位。",

"项目名称": "招采项目的正式名称。",

```
"预计采购时间": "预计进行（开始）采购活动的时间。",
"投标截止时间": "对潜在投标者开放报名的最后期限，或资格预审的截止期限。",
"招标（采购）单位联系人（非代理）": "招标（采购）单位的联系方式人员，不是代理机构联系人，非项目联系人",
"招标（采购）单位电话": "招标（采购）单位或招标单位联系人的联系电话。",
"代理机构联系人": "招标代理机构的联系人员或项目联系人，注意不是招标单位联系人。",
"代理机构电话": "招标代理机构或项目联系人的联系电话。",
"投标单位联系人": "参与投标的单位的联系人员。默认包含中标单位（供应商）联系人。",
"投标单位电话": "参与投标的单位的联系电话。",
"中标候选单位金额": "必须是原文中出现的中标候选单位提出的完成项目所需的金额，金额必须有单位（元、万元）。",
"最终中标单位联系人": "最终中标单位（供应商）的联系人员，不是项目联系人和代理机构联系人，注意区分。",
"最终中标单位电话": "最终中标单位（供应商）的联系电话。",
"招标文件位置": "可以获取到招标文件的位置。可能是具体地址、文件（doc、docx、pdf、zip）索引或文件路径。",
"订单编号": "采购订单的编号。",
"受文单位": "接收招标文件或合同的单位。",
"招标文件售价": "获取招标文件所需支付的费用，招标文件的售价。",
"投标保证金金额": "投标者需要缴纳的保证金金额，以确保投标的严肃性，金额必须有单位（元、万元）。"
}
```

#### #注意事项

1. "招标（采购）单位联系人（非代理）"和"代理机构联系人"是不一样的，注意区分。
2. 投标单位包括（大于等于）中标候选单位，中标候选单位包括（大于等于）中标单位。
3. "投标金额"和"中标候选单位金额"与"最终中标金额"是不一样的，注意区分。

## 数据后处理模块

在完成关键数据提取之后，为确保输出的数据能够被系统正确识别和使用，后处理步骤至关重要。数据后处理包括 JSON 格式标准化 和 数据格式化 两个部分，分别解决数据结构的完整性和数据内容的准确性问题。

### JSON 格式标准化

在使用大语言模型提取数据时，生成的 JSON 格式可能出现结构问题、不正确的语法、特殊字符等问题，导致数据无法正确解析。因此，需要通过 JSON 格式化工具对提取出的 JSON 数据进行标准化处理。[使用指南](#)

#### 参考代码



```
"""Utility functions for the OpenAI API."""
```

```
import json
import logging
import re
import ast
```

```
from json_repair import repair_json
```

```
log = logging.getLogger(__name__)
```

```
def try_parse_ast_to_json(function_string: str) -> tuple[str, dict]:
    """
    # 示例函数字符串
    function_string = "tool_call(first_int={'title': 'First Int', 'type': 'integer'
    :return:
    """
```

```
    tree = ast.parse(str(function_string).strip())
    ast_info = ""
    json_result = {}
    # 查找函数调用节点并提取信息
    for node in ast.walk(tree):
        if isinstance(node, ast.Call):
            function_name = node.func.id
            args = {kw.arg: kw.value for kw in node.keywords}
            ast_info += f"Function Name: {function_name}\r\n"
            for arg, value in args.items():
                ast_info += f"Argument Name: {arg}\n"
                ast_info += f"Argument Value: {ast.dump(value)}\n"
                json_result[arg] = ast.literal_eval(value)

    return ast_info, json_result
```

```
def try_parse_json_object(input: str) -> tuple[str, dict]:
```

```
result = None
try:
    # Try parse first
    result = json.loads(input)
except json.JSONDecodeError:
    log.info("Warning: Error decoding faulty json, attempting repair")

if result:
    return input, result

_pattern = r"\{(.*)\}"
_match = re.search(_pattern, input)
input = "{" + _match.group(1) + "}" if _match else input

# Clean up json string.
input = (
    input.replace("{", "{")
    .replace("}", "}")
    .replace('"{', "[{")
    .replace('}"', "}]")
    .replace("\\", " ")
    .replace("\\n", " ")
    .replace("\n", " ")
    .replace("\r", "")
    .strip()
)

# Remove JSON Markdown Frame
if input.startswith("```"):
    input = input[len("```"):]
if input.startswith("```json"):
    input = input[len("```json"):]
if input.endswith("```"):
    input = input[: len(input) - len("```")]

try:
    result = json.loads(input)
```



```
except json.JSONDecodeError:
    # Attempt to repair potentially malformed json string using json_repair.
    json_info = str(repair_json(json_str=input, return_objects=False))

    >
    # Generate JSON-string output using best-attempt prompting & parsing technique
    try:

        if len(json_info) < len(input):
            json_info, result = try_parse_ast_to_json(input)
        else:
            result = json.loads(json_info)

    except json.JSONDecodeError:
        log.exception("error loading json, json=%s", input)
        return json_info, {}
    else:
        if not isinstance(result, dict):
            log.exception("not expected dict type. type=%s:", type(result))
            return json_info, {}
        return json_info, result
else:
    return input, result
```

## 数据格式化

在确保 JSON 结构标准化后，还需要通过格式化工具对内容进行数据格式化。不同类型的数据，如日期、金额、文本等，需要遵循统一的格式要求。

- 日期格式：所有日期和时间字段都应格式化为标准的 14 位日期时间格式：**YYYYMMDDHHMMSS**。这可以确保时间字段在不同系统中具有一致的解析方式。
- 金额格式：金额字段应保留原单位（如元、万元），并且格式化为无空格、无额外字符的数值形式（如 **500000元**），以便在后续财务分析或报告生成中能够准确使用。
- 文本字段格式化：对文本字段中的特殊字符（如换行符、双引号）进行处理，确保文本内容不会破坏 JSON 的语法结构。比如，将双引号转义处理，或者移除无意义的换行符和空格。

如输入数据：

```
{  
  "项目名称": "智能楼宇工程",  
  "项目编号": "XZL-2023",  
  "采购预算": " 7,000,000.00 元",  
  "开标时间": "2024/01/01 09:00"  
}
```

格式化后的输出：

```
{  
  "项目名称": "智能楼宇工程",  
  "项目编号": "XZL-2023",  
  "采购预算": "7000000元",  
  "开标时间": "20240101090000"  
}
```

## 数据校验模块

校验模块是数据后处理过程中至关重要的一环。其作用是对最终的数据进行进一步的校验，确保数据的完整性、准确性和一致性。校验模块可以自动检测格式错误、逻辑冲突、缺失值等问题，并提供修复或警报机制。

### 格式校验

确保所有数据符合预期的格式标准，例如日期、金额、电话号码等字段的格式是否正确。

如：检查金额字段是否包含正确的货币单位，并确保数值的表示形式规范。

- 参考代码



```
def validate_currency_format(amount_str):  
    if '元' in amount_str or '万元' in amount_str:  
        try:  
            amount = float(amount_str.replace("万元", "").replace("元", "").replace("千", ""))  
            return True  
        except ValueError:  
            return False  
    return False
```

## 逻辑校验

逻辑校验是检查数据之间的逻辑关系是否符合业务规则。例如：

时间校验：投标截止时间不能晚于开标时间。校验时需检查两个时间字段，确保逻辑正确。

- 校验方法：比较投标截止时间和开标时间，如果投标截止时间晚于开标时间，则返回错误。
- 参考代码

```
from datetime import datetime  
  
def validate_time_order(submit_time, open_time):  
    submit_dt = datetime.strptime(submit_time, "%Y%m%d%H%M%S")  
    open_dt = datetime.strptime(open_time, "%Y%m%d%H%M%S")  
    return submit_dt <= open_dt
```

- 金额校验：采购预算金额不能小于中标金额。校验预算和中标金额，确保金额逻辑合理。
  - 校验方法：如果中标金额高于预算金额，则返回警报。

## 完整性校验

完整性校验确保所有关键字段都已经填入有效数据，避免信息缺失。对于未提供数据的字段，应填充默认值（如“无”），或触发错误提醒。

- 必填字段检查：对于某些字段，如“项目名称”、“项目编号”、“投标截止时间”，应强制要求填写，若缺失则进行标记或补全。





- 校验方法：通过预定义的字段列表检查 JSON 输出中是否包含所有必填字段。



- 自动填充默认值：如果某个字段为空或缺失，可以自动填充默认值“无”。

- 参考代码>

```
def fill_missing_fields(data, default="无"):
    required_fields = ["项目名称", "项目编号", "采购预算", "投标截止时间"]
    for field in required_fields:
        if field not in data or not data[field]:
            data[field] = default
    return data
```

## 一致性校验

一致性校验确保同一信息在不同字段或位置的值保持一致。例如：

- 项目编号一致性：项目编号在不同字段中应当相同，如出现在多个部分的项目编号不能出现不一致的情况。
  - 校验方法：检查项目编号是否一致，如果发现不同编号，则触发警报。
- 日期一致性：多个时间字段中如果是同一事件（如开始时间和结束时间在不同部分中重复出现），应确保其一致。
- 参考代码

```
def validate_data(json_data):  
    # 1. 格式校验  
    if not validate_date_format(json_data.get("投标截止时间", "")):  
        print("投标截止时间格式错误")  
    if not validate_currency_format(json_data.get("采购预算", "")):  
        print("采购预算格式错误")  
  
    # 2. 逻辑校验  
    if not validate_time_order(json_data.get("投标截止时间", ""), json_data.get("开标时间", "")):  
        print("投标截止时间不能晚于开标时间")  
  
    # 3. 完整性校验  
    json_data = fill_missing_fields(json_data)  
  
    # 4. 一致性校验  
    if json_data.get("项目编号") != json_data.get("计划编号"):  
        print("项目编号与计划编号不一致")  
  
    return json_data
```

## 数据修复模块

检测到数据格式或逻辑错误后，通过基于规则修复与更高级模型调用进行修复，确保数据的完整性和准确性。通过修复模块，能够自动纠正常见的错误，如格式错误、缺失数据或逻辑冲突，避免手动修正，提高效率。

### 基于规则的自动修复

在大部分情况下，错误可以通过预定义的规则和算法进行自动修复。此步骤作为第一层处理机制，针对格式错误、简单的逻辑冲突、特殊字符处理等问题进行修正。

- 格式修正：通过正则表达式或预定义算法修复日期、金额、电话号码等格式错误。
- 逻辑修正：检查和修复时间顺序、金额逻辑等问题。对投标截止时间、开标时间、金额关系进行简单调整。
- 数据填补：自动填补缺失字段，使用“无”或从其他字段推导合理值。

### 提交更高级模型处理

- 处理复杂业务逻辑：当多个数据字段之间存在复杂的依赖关系时，普通的规则引擎可能无法有效处理，例如合同条款中的复杂逻辑冲突，此时可以利用高级模型的上下文理解能力进行推理和调整。
- 识别与处理领域特定信息：高级模型擅长理解和处理特定领域的复杂术语、语境或结构不明的信息，如行业专用术语、合同中的特殊条款等。

## 数据处理神器-Batch API

Batch API 适用于无需即时反馈并需使用大模型处理大量请求的场景。通过 Batch API，开发者可以通过文件提交大量任务，且价格降低50%（GLM-4-Flash免费）、无并发限制。[Batch API 使用指南](#)

|     | 正常请求               | Batch请求            |
|-----|--------------------|--------------------|
| 任务量 | 1 亿请求（2048 tokens） | 1 亿请求（2048 tokens） |
| 模型  | GLM-4-Air          | GLM-4-Air          |
| 并发量 | 100 并发             | 4000 并发            |
| 天数  | 340 天              | 8.6 天 (40 倍效率)     |
| 价格  | 204,800 元          | 102,400 元（省钱一半）    |

## 单次处理千万级数据

| 模型          | Batch一次最大请求 |
|-------------|-------------|
| GLM-4-Flash | 1000万次      |
| GLM-4-Air   | 1000万次      |
| GLM-3-Turbo | 200万次       |
| Embedding-2 | 200万次       |
| Embedding-3 | 200万次       |

| 模型          | Batch一次最大请求 |
|-------------|-------------|
| BigModel    |             |
| GLM-4-Plus  | 200万次       |
| GLM-4-0520> | 50万次        |
| GLM-4       | 50万次        |

## 限时特惠资源包

GLM-4-AIR：卓越性能，性价比极高，高效处理海量数据，立即抢购：

- 1000万 GLM-4-AIR 推理资源包（3个月）：[立即购买](#)，限时特惠仅需3元

Embedding-3：全新升级，性能全面提升，支持自定义向量维度，限时优惠：

- 5000 万 Embedding-3 3 折尝鲜包（3 个月）：[立即购买](#)，限时特惠仅需 7.5 元

## 方案亮点

本方案的核心优势在于，它并不是试图以规则替代人工，而是通过引入大语言模型，**构建出一个真正“理解”招投标语境的智能提取系统**。从文件输入到结构化输出，每一步都围绕“准确提取”这个目标进行优化，而非仅仅满足格式转换。

方案在前端输入层就考虑到了现实复杂性，支持PDF、Word、HTML、扫描图像等格式，同时结合OCR与网页爬虫能力，确保信息不会在第一步就损失。预处理环节更是方案的基础支撑：它不是简单清洗噪声，而是对日期、金额、特殊符号、表格结构进行语义保留和规范化处理，为模型打好“地基”。

最关键的部分是Prompt工程，它不是泛泛而谈的“问答提示”，而是通过字段定义、异常处理策略、格式要求、输出模板等模块，逐步引导模型精准提取目标字段，确保输出数据的**稳定性与结构完整性**。哪怕遇到字段缺失或文档风格变化，系统也能以默认值、安全策略或异常提示机制，确保结果始终可落地。

此外，数据校验和修复机制不是附加模块，而是流程的一部分。格式是否合规、金额是否合理、字段是否一致，系统都会主动检查，并通过轻量规则或高级模型推理进行自动修正，大幅降低人工复核负担。

更值得一提的是，方案天然适配大批量数据场景。通过Batch API，每天处理上万条文档请求成为可能，不仅计算稳定，调用成本也极具性价比，适合长期、高频业务集成。

总体来看，这是一套既理解“数据”，也理解“业务”的工程化方案，它将人工智能的能力通过精密设计转化为可用、可靠、可规模化的数据抽取能力，真正服务于招投标信息管理这一传统而重要的行业场景。

[智能作文批改](#)

[数据分析](#)